PROJET FLUTTER

Tout le code

Main et fichiers associés

Quiz Game 7 • application Flutter

39 fichiers source/configuration • 16 fichiers Dart • 2 fichiers main.dart

Le point de démarrage Flutter est lib/main.dart ; le main.dart à la racine est également inclus.
Le PDF rassemble le code Dart, Android/Gradle, les scripts CI et la configuration du projet.

Périmètre du document

- Inclus : tous les fichiers Dart suivis, les deux fichiers main.dart, les sources Android, les scripts de build et les configurations suivies.
- Non inclus : les JSON des questions/niveaux, les images et sons (ressources, pas du code), les caches/métadonnées générés et android/local.properties.
- android/local.properties est volontairement omis car il contient des chemins propres à un ordinateur (SDK Flutter/Android).
- Les numéros en marge ne font pas partie du code ; les symboles non couverts sont notés \u{…} dans le PDF, sans modifier les fichiers source.

Branche : arena/11f2f6d3-test

Édition du 17 octobre 2026

Table des matières

39 fichiers source/configuration

Flutter et Dart

lib/main.dart 3 • 35 l.
lib/models/question.dart 4 • 80 l.
lib/models/quiz_category.dart 5 • 76 l.
lib/models/user_profile.dart 6 • 34 l.
lib/screens/level_map_screen.dart 7 • 249 l.
lib/screens/quiz_screen.dart 10 • 320 l.
lib/screens/quiz_type_selection_screen.dart 14 • 126 l.
lib/screens/score_screen.dart 16 • 138 l.
lib/screens/scores_screen.dart 18 • 131 l.
lib/services/level_manager.dart 20 • 133 l.
lib/services/profile_functions.dart 22 • 36 l.
lib/services/question_loader.dart 23 • 107 l.
lib/services/quiz_scoring.dart 25 • 15 l.
lib/widgets/animated_background.dart 26 • 32 l.
lib/widgets/cycling_background.dart 27 • 78 l.
main.dart 28 • 20 l.

Scripts et compilation automatisée

.github/scripts/fix_android_config.py 47 • 113 l.
.github/scripts/prepare_project.py 49 • 339 l.
.github/workflows/build-apk.yml 53 • 209 l.

Android et Gradle

android/app/build.gradle.kts 29 • 42 l.
android/app/src/debug/AndroidManifest.xml 30 • 7 l.
android/app/src/main/AndroidManifest.xml 31 • 45 l.
android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java 32 • 34 l.
android/app/src/main/kotlin/com/example/quiz_game/MainActivity.kt 33 • 5 l.
android/app/src/main/kotlin/com/example/quiz_game7/MainActivity.kt 34 • 5 l.
android/app/src/main/res/drawable-v21/launch_background.xml 35 • 12 l.
android/app/src/main/res/drawable/launch_background.xml 36 • 12 l.
android/app/src/main/res/values-night/styles.xml 37 • 18 l.
android/app/src/main/res/values/styles.xml 38 • 18 l.
android/app/src/profile/AndroidManifest.xml 39 • 7 l.
android/build.gradle.kts 40 • 21 l.
android/gradle.properties 41 • 3 l.
android/gradle/wrapper/gradle-wrapper.properties 42 • 5 l.
android/gradlew 43 • 160 l.
android/gradlew.bat 45 • 90 l.
android/settings.gradle.kts 46 • 25 l.

Configuration du projet

.gitignore 56 • 45 l.
analysis_options.yaml 57 • 31 l.
pubspec.yaml 58 • 67 l.

Ouvrir un fichier : repérez son numéro de page, puis consultez les pages suivantes s’il occupe plusieurs pages.

lib/main.dart

35 lignes

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/screens/quiz_type_selection_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const QuizGameApp());
}

class QuizGameApp extends StatelessWidget {
  const QuizGameApp({super.key});

  @override
  Widget build(BuildContext context) {
    const seedColor = Color(0xFF493A8C);
    return MaterialApp(
      title: 'Quiz éducatif',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: seedColor),
        appBarTheme: const AppBarTheme(
          backgroundColor: Color(0xFF182848),
          foregroundColor: Colors.white,
          centerTitle: true,
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            minimumSize: const Size.fromHeight(48),
          ),
        ),
      ),
      home: const QuizTypeSelectionScreen(),
    );
  }
}
```

```dart
import 'dart:math';

class Question {
  Question({
    required String question,
    required String correctAnswer,
    required List<String> options,
  })  : question = question.trim(),
        correctAnswer = correctAnswer.trim(),
        options = List<String>.unmodifiable(
          options.map((option) => option.trim()),
        ) {
    if (this.question.isEmpty) {
      throw ArgumentError.value(question, 'question', 'ne peut pas être vide');
    }
    if (this.correctAnswer.isEmpty) {
      throw ArgumentError.value(
        correctAnswer,
        'correctAnswer',
        'ne peut pas être vide',
      );
    }
    if (this.options.length < 2) {
      throw ArgumentError.value(options, 'options', 'il en faut au moins deux');
    }
    if (this.options.any((option) => option.isEmpty)) {
      throw ArgumentError.value(options, 'options', 'ne peut pas contenir de réponse vide');
    }
    if (this.options.toSet().length != this.options.length) {
      throw ArgumentError.value(options, 'options', 'ne peut pas contenir de doublons');
    }
    if (!this.options.contains(this.correctAnswer)) {
      throw ArgumentError.value(
        correctAnswer,
        'correctAnswer',
        'doit être présente dans les options',
      );
    }
  }

  final String question;
  final String correctAnswer;
  final List<String> options;

  factory Question.fromJson(Map<String, dynamic> json) {
    final rawQuestion = json['question'] ?? json['arabe'];
    final rawCorrectAnswer = json['correct'] ?? json['correctAnswer'];
    final rawOptions = json['options'];

    if (rawQuestion is! String || rawQuestion.trim().isEmpty) {
      throw const FormatException('La question doit être une chaîne non vide.');
    }
    if (rawCorrectAnswer is! String || rawCorrectAnswer.trim().isEmpty) {
      throw const FormatException('La bonne réponse doit être une chaîne non vide.');
    }
    if (rawOptions is! List || rawOptions.any((option) => option is! String)) {
      throw const FormatException('Le champ « options » doit être une liste de textes.');
    }

    // Les fichiers de questions listent souvent la bonne réponse en premier :
    // on mélange l'ordre à chaque chargement pour éviter toute position fixe.
    final shuffledOptions = rawOptions.cast<String>().toList()..shuffle(Random());

    try {
      return Question(
        question: rawQuestion,
        correctAnswer: rawCorrectAnswer,
        options: shuffledOptions,
      );
    } on ArgumentError catch (error) {
      throw FormatException('Question invalide : $error');
    }
  }

  Map<String, dynamic> toJson() => <String, dynamic>{
        'question': question,
        'correct': correctAnswer,
        'options': options,
      };
}
```

```dart
import 'package:flutter/material.dart';

@immutable
class QuizCategory {
  const QuizCategory({
    required this.id,
    required this.label,
    required this.description,
    required this.icon,
  });

  /// Stable id used for preference keys and asset directories.
  final String id;
  final String label;
  final String description;
  final IconData icon;
}

const List<QuizCategory> quizCategories = <QuizCategory>[
  QuizCategory(
    id: 'arabe',
    label: 'Arabe',
    description: 'Vocabulaire et langue arabe',
    icon: Icons.translate,
  ),
  QuizCategory(
    id: 'sira',
    label: 'Sira',
    description: 'La vie du Prophète',
    icon: Icons.book_outlined,
  ),
  QuizCategory(
    id: 'coran',
    label: 'Coran',
    description: 'Connaissances autour du Coran',
    icon: Icons.menu_book,
  ),
  QuizCategory(
    id: 'histoire',
    label: 'Histoire',
    description: 'Repères historiques',
    icon: Icons.account_balance,
  ),
  QuizCategory(
    id: 'sahaba',
    label: 'Sahaba',
    description: 'Les compagnons',
    icon: Icons.people_alt,
  ),
  QuizCategory(
    id: 'hijra',
    label: 'Hijra',
    description: 'Le récit de l’Hégire',
    icon: Icons.route,
  ),
  QuizCategory(
    id: 'terme',
    label: 'Termes',
    description: 'Mots et notions à connaître',
    icon: Icons.lightbulb_outline,
  ),
  QuizCategory(
    id: 'general',
    label: 'Général',
    description: 'Questions variées',
    icon: Icons.quiz,
  ),
];

QuizCategory? quizCategoryForId(String id) {
  final normalizedId = id.trim().toLowerCase();
  for (final category in quizCategories) {
    if (category.id == normalizedId) return category;
  }
  return null;
}
```

lib/models/user_profile.dart

34 lignes

```dart
class UserProfile {
  UserProfile({required this.name, required this.avatar, this.score = 0});

  final String name;
  final String avatar;
  int score;

  Map<String, dynamic> toJson() => <String, dynamic>{
        'name': name,
        'avatar': avatar,
        'score': score,
      };

  factory UserProfile.fromJson(Map<String, dynamic> json) {
    final rawScore = json['score'];
    final score = rawScore is num
        ? rawScore.toInt()
        : int.tryParse(rawScore?.toString() ?? '') ?? 0;

    return UserProfile(
      name: json['name']?.toString() ?? '',
      avatar: json['avatar']?.toString() ?? '\u{1F642}',
      score: score < 0 ? 0 : score,
    );
  }

  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is UserProfile && name == other.name && avatar == other.avatar;

  @override
  int get hashCode => Object.hash(name, avatar);
}
```

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/models/quiz_category.dart';
import 'package:quiz_game7/screens/quiz_screen.dart';
import 'package:quiz_game7/services/level_manager.dart';
import 'package:quiz_game7/services/question_loader.dart';
import 'package:quiz_game7/widgets/animated_background.dart';

class LevelMapScreen extends StatefulWidget {
  const LevelMapScreen({super.key, required this.quizType});

  final String quizType;

  @override
  State<LevelMapScreen> createState() => _LevelMapScreenState();
}

class _LevelMapScreenState extends State<LevelMapScreen> {
  List<int> _availableLevels = <int>[];
  Set<int> _unlockedLevels = <int>{1};
  bool _isLoading = true;
  String? _errorMessage;

  String get _title => quizCategoryForId(widget.quizType)?.label ?? widget.quizType;

  @override
  void initState() {
    super.initState();
    _loadLevels();
  }

  Future<void> _loadLevels() async {
    if (mounted) {
      setState(() {
        _isLoading = true;
        _errorMessage = null;
      });
    }

    try {
      final available = await QuestionLoader.loadAvailableLevels(widget.quizType);
      if (available.isNotEmpty) {
        await LevelManager.initializeLevelsForQuiz(
          widget.quizType,
          totalLevels: available.last,
        );
      }
      final unlockedStrings = await LevelManager.getUnlockedLevels(widget.quizType);
      if (!mounted) return;
      setState(() {
        _availableLevels = available;
        _unlockedLevels = unlockedStrings.map(int.parse).toSet();
        _isLoading = false;
      });
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = error.toString();
      });
    }
  }

  Future<void> _startLevel(int level) async {
    final completed = await Navigator.of(context).push<bool>(
      MaterialPageRoute<bool>(
        builder: (_) => QuizScreen(
          niveau: level,
          quizType: widget.quizType,
          availableLevels: List<int>.unmodifiable(_availableLevels),
        ),
      ),
    );
    if (completed == true) await _loadLevels();
  }

  Future<void> _confirmReset() async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Réinitialiser la progression ?'),
        content: Text('Les niveaux débloqués pour « $_title » seront remis à zéro.'),
        actions: <Widget>[
          TextButton(
            onPressed: () => Navigator.of(context).pop(false),
            child: const Text('Annuler'),
          ),
          FilledButton(
            onPressed: () => Navigator.of(context).pop(true),
            child: const Text('Réinitialiser'),
          ),
```

```
        ],
      ),
    );
    if (confirmed != true || !mounted) return;

    try {
      await LevelManager.resetLevels(
        widget.quizType,
        totalLevels: _availableLevels.last,
      );
      await _loadLevels();
    } catch (error) {
      if (!mounted) return;
      setState(() => _errorMessage = error.toString());
    }
  }

  Widget _buildLevelList() {
    if (_isLoading) {
      return const Expanded(
        child: Center(child: CircularProgressIndicator(color: Colors.white)),
      );
    }

    if (_errorMessage != null) {
      return Expanded(
        child: Center(
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: <Widget>[
                const Icon(Icons.error_outline, color: Colors.white, size: 48),
                const SizedBox(height: 12),
                Text(
                  _errorMessage!,
                  textAlign: TextAlign.center,
                  style: const TextStyle(color: Colors.white),
                ),
                const SizedBox(height: 16),
                OutlinedButton.icon(
                  onPressed: _loadLevels,
                  icon: const Icon(Icons.refresh),
                  label: const Text('Réessayer'),
                ),
              ],
            ),
          ),
        ),
      );
    }

    if (_availableLevels.isEmpty) {
      return const Expanded(
        child: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: <Widget>[
                Icon(Icons.inbox_outlined, color: Colors.white, size: 52),
                SizedBox(height: 12),
                Text(
                  'Aucun niveau n’est encore configuré pour ce thème.',
                  textAlign: TextAlign.center,
                  style: TextStyle(color: Colors.white, fontSize: 18),
                ),
                SizedBox(height: 8),
                Text(
                  'Ajoutez des numéros dans le fichier levels.json pour activer le quiz.',
                  textAlign: TextAlign.center,
                  style: TextStyle(color: Colors.white70),
                ),
              ],
            ),
          ),
        ),
      );
    }

    return Expanded(
      child: ListView.separated(
        padding: const EdgeInsets.fromLTRB(16, 0, 16, 20),
        itemCount: _availableLevels.length,
        separatorBuilder: (_, __) => const SizedBox(height: 8),
        itemBuilder: (context, index) {
          final level = _availableLevels[index];
          final isUnlocked = _unlockedLevels.contains(level);
          return Card(
            color: isUnlocked ? Colors.white : Colors.white.withOpacity(0.82),
```

```
            child: ListTile(
              leading: CircleAvatar(
                backgroundColor: isUnlocked ? const Color(0xFF493A8C) : Colors.grey,
                child: Icon(
                  isUnlocked ? Icons.lock_open : Icons.lock_outline,
                  color: Colors.white,
                ),
              ),
              title: Text(
                'Niveau $level',
                style: const TextStyle(fontWeight: FontWeight.bold),
              ),
              subtitle: Text(isUnlocked ? 'Touchez pour jouer' : 'Terminez le niveau précédent'),
              trailing: Icon(isUnlocked ? Icons.play_arrow : Icons.lock),
              enabled: isUnlocked,
              onTap: isUnlocked ? () => _startLevel(level) : null,
            ),
          );
        },
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(_title),
        actions: <Widget>[
          IconButton(
            tooltip: 'Réinitialiser la progression',
            onPressed: _isLoading || _availableLevels.isEmpty ? null : _confirmReset,
            icon: const Icon(Icons.restart_alt),
          ),
        ],
      ),
      body: AnimatedBackground(
        child: SafeArea(
          child: Column(
            children: <Widget>[
              Padding(
                padding: const EdgeInsets.fromLTRB(20, 18, 20, 16),
                child: Column(
                  children: <Widget>[
                    const Icon(Icons.school_outlined, color: Colors.white, size: 50),
                    const SizedBox(height: 8),
                    Text(
                      'Choisissez un niveau pour le thème « $_title ».',
                      textAlign: TextAlign.center,
                      style: const TextStyle(color: Colors.white, fontSize: 18),
                    ),
                    if (_availableLevels.isNotEmpty) ...<Widget>[
                      const SizedBox(height: 8),
                      Text(
                        '${_unlockedLevels.intersection(_availableLevels.toSet()).length} / ${_availableLevels.length} niveaux débloqués',
                        style: const TextStyle(color: Colors.white70),
                      ),
                    ],
                  ],
                ),
              ),
              _buildLevelList(),
            ],
          ),
        ),
      ),
    );
  }
}
```

```dart
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:quiz_game7/models/question.dart';
import 'package:quiz_game7/models/quiz_category.dart';
import 'package:quiz_game7/screens/score_screen.dart';
import 'package:quiz_game7/services/level_manager.dart';
import 'package:quiz_game7/services/profile_functions.dart';
import 'package:quiz_game7/services/question_loader.dart';
import 'package:quiz_game7/services/quiz_scoring.dart';
import 'package:quiz_game7/widgets/animated_background.dart';

class QuizScreen extends StatefulWidget {
  const QuizScreen({
    super.key,
    required this.niveau,
    required this.quizType,
    required this.availableLevels,
    this.secondsPerQuestion = 10,
  }) : assert(secondsPerQuestion > 0);

  final int niveau;
  final String quizType;
  final List<int> availableLevels;
  final int secondsPerQuestion;

  @override
  State<QuizScreen> createState() => _QuizScreenState();
}

class _QuizScreenState extends State<QuizScreen> with WidgetsBindingObserver {
  List<Question> _questions = <Question>[];
  List<String?> _answers = <String?>[];
  int _currentIndex = 0;
  late int _timeLeft = widget.secondsPerQuestion;
  Timer? _timer;
  bool _isLoading = true;
  bool _isFinishing = false;
  String? _errorMessage;

  String get _title => quizCategoryForId(widget.quizType)?.label ?? widget.quizType;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _loadQuestions();
  }

  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.resumed) {
      if (!_isLoading && _errorMessage == null && _questions.isNotEmpty && !_isFinishing) {
        _startTimer(reset: false);
      }
    } else {
      _timer?.cancel();
      _timer = null;
    }
  }

  Future<void> _loadQuestions() async {
    if (mounted) {
      setState(() {
        _isLoading = true;
        _errorMessage = null;
      });
    }

    try {
      final questions = await QuestionLoader.loadQuestions(
        widget.quizType,
        widget.niveau,
      );
      if (!mounted) return;
      setState(() {
        _questions = questions;
        _answers = List<String?>.filled(questions.length, null);
        _currentIndex = 0;
        _timeLeft = widget.secondsPerQuestion;
        _isLoading = false;
      });
      _startTimer();
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = error.toString();
      });
    }
```

```
  }

  void _startTimer({bool reset = true}) {
    _timer?.cancel();
    if (reset) _timeLeft = widget.secondsPerQuestion;
    _timer = Timer.periodic(const Duration(seconds: 1), (_) {
      if (!mounted || _isFinishing) return;
      if (_timeLeft <= 1) {
        _timer?.cancel();
        setState(() => _timeLeft = 0);
        _submitAnswer(null);
      } else {
        setState(() => _timeLeft--);
      }
    });
  }

  void _submitAnswer(String? answer) {
    if (_isFinishing || _isLoading || _questions.isEmpty) return;
    _timer?.cancel();

    final isLastQuestion = _currentIndex == _questions.length - 1;
    setState(() {
      _answers[_currentIndex] = answer;
      if (isLastQuestion) {
        _isFinishing = true;
      } else {
        _currentIndex++;
        _timeLeft = widget.secondsPerQuestion;
      }
    });

    if (isLastQuestion) {
      unawaited(_finishQuiz());
    } else {
      _startTimer();
    }
  }

  Future<void> _finishQuiz() async {
    final score = calculateQuizScore(_questions, _answers);
    int? unlockedNextLevel;
    final persistenceErrors = <String>[];

    try {
      await ProfileManager.updateScore(score);
    } catch (error) {
      persistenceErrors.add('Le score n’a pas pu être enregistré : $error');
    }
    try {
      unlockedNextLevel = await LevelManager.unlockNextLevel(
        widget.niveau,
        widget.quizType,
        availableLevels: widget.availableLevels,
      );
    } catch (error) {
      persistenceErrors.add('La progression n’a pas pu être enregistrée : $error');
    }

    if (!mounted) return;
    Navigator.of(context).pushReplacement<void, bool>(
      MaterialPageRoute<void>(
        builder: (_) => ScoreScreen(
          quizType: widget.quizType,
          niveau: widget.niveau,
          questions: List<Question>.unmodifiable(_questions),
          answers: List<String?>.unmodifiable(_answers),
          unlockedNextLevel: unlockedNextLevel,
          persistenceError: persistenceErrors.isEmpty
              ? null
              : persistenceErrors.join('\n'),
        ),
      ),
      result: true,
    );
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _timer?.cancel();
    super.dispose();
  }

  Widget _buildMessage(String message, {required bool isError}) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
```

```
          children: <Widget>[
            Icon(
              isError ? Icons.error_outline : Icons.hourglass_top,
              color: Colors.white,
              size: 48,
            ),
            const SizedBox(height: 12),
            Text(
              message,
              textAlign: TextAlign.center,
              style: const TextStyle(color: Colors.white, fontSize: 16),
            ),
            if (isError) ...<Widget>[
              const SizedBox(height: 16),
              OutlinedButton.icon(
                onPressed: _loadQuestions,
                icon: const Icon(Icons.refresh),
                label: const Text('Réessayer'),
              ),
            ],
          ],
        ),
      ),
    );
  }

  Widget _buildQuiz() {
    final question = _questions[_currentIndex];
    final quizProgress = (_currentIndex + 1) / _questions.length;
    final timeProgress = _timeLeft / widget.secondsPerQuestion;
    final currentScore = calculateQuizScore(_questions, _answers);

    return Column(
      children: <Widget>[
        Padding(
          padding: const EdgeInsets.fromLTRB(20, 16, 20, 12),
          child: Column(
            children: <Widget>[
              Text(
                'Question ${_currentIndex + 1} sur ${_questions.length}',
                style: const TextStyle(color: Colors.white, fontSize: 16),
              ),
              const SizedBox(height: 6),
              LinearProgressIndicator(value: quizProgress),
              const SizedBox(height: 14),
              Row(
                children: <Widget>[
                  const Icon(Icons.emoji_events_outlined, color: Colors.amber),
                  const SizedBox(width: 6),
                  Text(
                    'Score : $currentScore',
                    style: const TextStyle(color: Colors.white),
                  ),
                  const Spacer(),
                  Text(
                    '$_timeLeft s',
                    style: TextStyle(
                      color: _timeLeft <= 3 ? Colors.redAccent : Colors.white,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 6),
              Semantics(
                label: 'Temps restant',
                child: LinearProgressIndicator(
                  value: timeProgress.clamp(0.0, 1.0).toDouble(),
                  color: _timeLeft <= 3 ? Colors.redAccent : Colors.lightGreenAccent,
                  backgroundColor: Colors.white24,
                ),
              ),
            ],
          ),
        ),
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.fromLTRB(20, 12, 20, 24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: <Widget>[
                Card(
                  color: Colors.white,
                  child: Padding(
                    padding: const EdgeInsets.all(20),
                    child: Text(
                      question.question,
                      textAlign: TextAlign.center,
                      style: const TextStyle(
                        fontSize: 21,
```

```
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                ),
                const SizedBox(height: 12),
                for (final option in question.options) ...<Widget>[
                  Padding(
                    padding: const EdgeInsets.symmetric(vertical: 5),
                    child: ElevatedButton(
                      onPressed: _isFinishing ? null : () => _submitAnswer(option),
                      style: ElevatedButton.styleFrom(
                        backgroundColor: Colors.white,
                        foregroundColor: const Color(0xFF34266D),
                        padding: const EdgeInsets.symmetric(
                          horizontal: 16,
                          vertical: 14,
                        ),
                        textStyle: const TextStyle(fontSize: 16),
                      ),
                      child: Text(option, textAlign: TextAlign.center),
                    ),
                  ),
                ],
              ],
            ),
          ),
        ),
      ],
    );
  }

  @override
  Widget build(BuildContext context) {
    final body = _isLoading
        ? const Center(child: CircularProgressIndicator(color: Colors.white))
        : _errorMessage != null
            ? _buildMessage(_errorMessage!, isError: true)
            : _questions.isEmpty
                ? _buildMessage('Aucune question n’est disponible pour ce niveau.', isError: true)
                : _buildQuiz();

    return Scaffold(
      appBar: AppBar(title: Text('$_title — niveau ${widget.niveau}')),
      body: AnimatedBackground(
        child: SafeArea(child: body),
      ),
    );
  }
}
```

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/models/quiz_category.dart';
import 'package:quiz_game7/screens/level_map_screen.dart';
import 'package:quiz_game7/screens/scores_screen.dart';
import 'package:quiz_game7/widgets/animated_background.dart';

class QuizTypeSelectionScreen extends StatelessWidget {
  const QuizTypeSelectionScreen({super.key});

  void _openScores(BuildContext context) {
    Navigator.of(context).push<void>(
      MaterialPageRoute<void>(builder: (_) => const ScoresScreen()),
    );
  }

  void _openCategory(BuildContext context, QuizCategory category) {
    Navigator.of(context).push<void>(
      MaterialPageRoute<void>(
        builder: (_) => LevelMapScreen(quizType: category.id),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Quiz éducatif'),
        actions: <Widget>[
          IconButton(
            tooltip: 'Voir les scores',
            onPressed: () => _openScores(context),
            icon: const Icon(Icons.emoji_events_outlined),
          ),
        ],
      ),
      body: AnimatedBackground(
        child: SafeArea(
          child: LayoutBuilder(
            builder: (context, constraints) {
              final columns = constraints.maxWidth >= 700 ? 4 : 2;
              return Column(
                children: <Widget>[
                  const Padding(
                    padding: EdgeInsets.fromLTRB(24, 24, 24, 8),
                    child: Text(
                      'Choisissez un thème',
                      textAlign: TextAlign.center,
                      style: TextStyle(
                        color: Colors.white,
                        fontSize: 26,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                  ),
                  const Padding(
                    padding: EdgeInsets.symmetric(horizontal: 24),
                    child: Text(
                      'Apprenez en jouant, à votre rythme.',
                      textAlign: TextAlign.center,
                      style: TextStyle(color: Colors.white70, fontSize: 16),
                    ),
                  ),
                  Expanded(
                    child: GridView.builder(
                      padding: const EdgeInsets.all(20),
                      itemCount: quizCategories.length,
                      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
                        crossAxisCount: columns,
                        crossAxisSpacing: 14,
                        mainAxisSpacing: 14,
                        childAspectRatio: columns == 2 ? 0.9 : 0.95,
                      ),
                      itemBuilder: (context, index) {
                        final category = quizCategories[index];
                        return Card(
                          color: Colors.white,
                          clipBehavior: Clip.antiAlias,
                          shape: RoundedRectangleBorder(
                            borderRadius: BorderRadius.circular(20),
                          ),
                          child: InkWell(
                            onTap: () => _openCategory(context, category),
                            child: Padding(
                              padding: const EdgeInsets.all(12),
                              child: Column(
                                mainAxisAlignment: MainAxisAlignment.center,
                                children: <Widget>[
                                  Icon(category.icon, size: 42, color: const Color(0xFF493A8C)),
                                  const SizedBox(height: 8),
```

```
                                Text(
                                  category.label,
                                  textAlign: TextAlign.center,
                                  style: const TextStyle(
                                    fontSize: 18,
                                    fontWeight: FontWeight.bold,
                                  ),
                                ),
                                const SizedBox(height: 4),
                                Text(
                                  category.description,
                                  textAlign: TextAlign.center,
                                  maxLines: 2,
                                  overflow: TextOverflow.ellipsis,
                                  style: TextStyle(
                                    color: Colors.grey.shade700,
                                    fontSize: 12,
                                  ),
                                ),
                              ],
                            ),
                          ),
                        ),
                      ),
                    );
                  },
                ),
              ],
            );
          },
        ),
      ),
    );
  }
}
```

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/models/question.dart';
import 'package:quiz_game7/models/quiz_category.dart';
import 'package:quiz_game7/services/quiz_scoring.dart';
import 'package:quiz_game7/widgets/animated_background.dart';

class ScoreScreen extends StatelessWidget {
  const ScoreScreen({
    super.key,
    required this.quizType,
    required this.niveau,
    required this.questions,
    required this.answers,
    this.unlockedNextLevel,
    this.persistenceError,
  });

  final String quizType;
  final int niveau;
  final List<Question> questions;
  final List<String?> answers;
  final int? unlockedNextLevel;
  final String? persistenceError;

  @override
  Widget build(BuildContext context) {
    final score = calculateQuizScore(questions, answers);
    final category = quizCategoryForId(quizType)?.label ?? quizType;
    final isPerfect = score == questions.length && questions.isNotEmpty;

    return Scaffold(
      appBar: AppBar(title: const Text('Résultat du quiz')),
      body: AnimatedBackground(
        child: SafeArea(
          child: Column(
            children: <Widget>[
              Padding(
                padding: const EdgeInsets.fromLTRB(20, 20, 20, 12),
                child: Column(
                  children: <Widget>[
                    Icon(
                      isPerfect ? Icons.celebration : Icons.emoji_events,
                      color: Colors.amber,
                      size: 54,
                    ),
                    const SizedBox(height: 8),
                    Text(
                      '$score / ${questions.length}',
                      style: const TextStyle(
                        color: Colors.white,
                        fontSize: 32,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                    Text(
                      '$category — niveau $niveau',
                      style: const TextStyle(color: Colors.white70, fontSize: 16),
                    ),
                    if (unlockedNextLevel != null) ...<Widget>[
                      const SizedBox(height: 8),
                      Text(
                        'Niveau $unlockedNextLevel débloqué !',
                        style: const TextStyle(
                          color: Colors.lightGreenAccent,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                    ],
                    if (persistenceError != null) ...<Widget>[
                      const SizedBox(height: 8),
                      Text(
                        persistenceError!,
                        textAlign: TextAlign.center,
                        style: const TextStyle(color: Colors.orangeAccent),
                      ),
                    ],
                  ],
                ),
              ),
              Expanded(
                child: ListView.separated(
                  padding: const EdgeInsets.fromLTRB(16, 4, 16, 12),
                  itemCount: questions.length,
                  separatorBuilder: (_, __) => const SizedBox(height: 8),
                  itemBuilder: (context, index) {
                    final question = questions[index];
                    final answer = index < answers.length ? answers[index] : null;
                    final displayAnswer = answer ?? 'Aucune réponse';
                    final isCorrect = answer == question.correctAnswer;
                    return Card(
```

```
                    color: Colors.white,
                    child: Padding(
                      padding: const EdgeInsets.all(12),
                      child: Row(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: <Widget>[
                          Icon(
                            isCorrect ? Icons.check_circle : Icons.cancel,
                            color: isCorrect ? Colors.green : Colors.redAccent,
                          ),
                          const SizedBox(width: 10),
                          Expanded(
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: <Widget>[
                                Text(
                                  question.question,
                                  style: const TextStyle(fontWeight: FontWeight.bold),
                                ),
                                const SizedBox(height: 4),
                                Text('Votre réponse : $displayAnswer'),
                                if (!isCorrect)
                                  Text('Bonne réponse : ${question.correctAnswer}'),
                              ],
                            ),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 4, 20, 16),
              child: FilledButton.icon(
                onPressed: () => Navigator.of(context).pop(),
                icon: const Icon(Icons.map_outlined),
                label: const Text('Retour aux niveaux'),
              ),
            ),
          ],
        ),
      ),
        ),
      );
    }
}
```

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/services/profile_functions.dart';
import 'package:quiz_game7/widgets/animated_background.dart';

class ScoresScreen extends StatefulWidget {
  const ScoresScreen({super.key});

  @override
  State<ScoresScreen> createState() => _ScoresScreenState();
}

class _ScoresScreenState extends State<ScoresScreen> {
  int _totalScore = 0;
  bool _isLoading = true;
  String? _errorMessage;

  @override
  void initState() {
    super.initState();
    _loadScore();
  }

  Future<void> _loadScore() async {
    try {
      final total = await ProfileManager.getTotalScore();
      if (!mounted) return;
      setState(() {
        _totalScore = total;
        _isLoading = false;
        _errorMessage = null;
      });
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = error.toString();
      });
    }
  }

  Future<void> _resetScore() async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Effacer le score ?'),
        content: const Text('Cette action remettra votre score total à zéro.'),
        actions: <Widget>[
          TextButton(
            onPressed: () => Navigator.of(context).pop(false),
            child: const Text('Annuler'),
          ),
          FilledButton(
            onPressed: () => Navigator.of(context).pop(true),
            child: const Text('Effacer'),
          ),
        ],
      ),
    );
    if (confirmed != true || !mounted) return;

    try {
      await ProfileManager.resetScore();
      await _loadScore();
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = 'Impossible de réinitialiser le score : $error';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tous les scores')),
      body: AnimatedBackground(
        child: SafeArea(
          child: Center(
            child: _isLoading
                ? const CircularProgressIndicator(color: Colors.white)
                : _errorMessage != null
                    ? Padding(
                        padding: const EdgeInsets.all(24),
                        child: Text(
                          'Impossible de charger le score : $_errorMessage',
                          textAlign: TextAlign.center,
                          style: const TextStyle(color: Colors.white),
                        ),
                      )
```

```
                    : Padding(
                        padding: const EdgeInsets.all(24),
                        child: Column(
                          mainAxisSize: MainAxisSize.min,
                          children: <Widget>[
                            const Icon(
                              Icons.emoji_events,
                              color: Colors.amber,
                              size: 72,
                            ),
                            const SizedBox(height: 12),
                            const Text(
                              'Votre score total',
                              style: TextStyle(color: Colors.white, fontSize: 20),
                            ),
                            const SizedBox(height: 6),
                            Text(
                              '$_totalScore points',
                              style: const TextStyle(
                                color: Colors.white,
                                fontSize: 34,
                                fontWeight: FontWeight.bold,
                              ),
                            ),
                            if (_totalScore > 0) ...<Widget>[
                              const SizedBox(height: 20),
                              OutlinedButton.icon(
                                onPressed: _resetScore,
                                icon: const Icon(Icons.delete_outline),
                                label: const Text('Réinitialiser le score'),
                              ),
                            ],
                          ],
                        ),
                      ),
            ),
          ),
        ),
      );
    }
}
```

```dart
import 'package:shared_preferences/shared_preferences.dart';

class LevelManager {
  static String _keyFor(String quizType) {
    final normalizedType = quizType.trim().toLowerCase();
    if (!RegExp(r'^[a-z0-9_-]+$').hasMatch(normalizedType)) {
      throw ArgumentError.value(quizType, 'quizType', 'identifiant invalide');
    }
    return 'levels__$normalizedType';
  }

  /// Starts a quiz with only its first level unlocked.
  static Future<void> initializeLevelsForQuiz(
    String quizType, {
    int totalLevels = 10,
  }) async {
    if (totalLevels < 1) {
      throw ArgumentError.value(totalLevels, 'totalLevels', 'doit être positif');
    }

    final prefs = await SharedPreferences.getInstance();
    final key = _keyFor(quizType);
    final hadCanonicalValue = prefs.containsKey(key);
    var stored = prefs.getStringList(key);

    // Preserve progress written by the older capitalized "Terme" category id.
    final trimmedType = quizType.trim();
    final titleCaseType = trimmedType.isEmpty
        ? trimmedType
        : '${trimmedType[0].toUpperCase()}${trimmedType.substring(1).toLowerCase()}';
    final legacyKeys = <String>{
      'levels__$trimmedType',
      'levels__$titleCaseType',
    };
    if (stored == null) {
      for (final legacyKey in legacyKeys) {
        if (legacyKey == key) continue;
        stored = prefs.getStringList(legacyKey);
        if (stored != null) {
          await prefs.remove(legacyKey);
          break;
        }
      }
    }

    final validLevels = <int>{};
    for (final value in stored ?? const <String>[]) {
      final parsed = int.tryParse(value);
      if (parsed != null && parsed >= 1 && parsed <= totalLevels) {
        validLevels.add(parsed);
      }
    }
    validLevels.add(1);
    final sorted = validLevels.toList()..sort();
    final normalized = sorted.map((level) => level.toString()).toList();
    if (!hadCanonicalValue || stored == null || stored.join(',') != normalized.join(',')) {
      await prefs.setStringList(key, normalized);
    }
  }

  /// Unlocks the next configured level after the current one is completed.
  /// Returns its number, or null when there is no next level to unlock.
  static Future<int?> unlockNextLevel(
    int completedLevel,
    String quizType, {
    required List<int> availableLevels,
  }) async {
    if (completedLevel < 1) return null;

    final levels = availableLevels.where((level) => level > 0).toSet().toList()
      ..sort();
    final currentIndex = levels.indexOf(completedLevel);
    if (currentIndex == -1 || currentIndex + 1 >= levels.length) return null;

    final prefs = await SharedPreferences.getInstance();
    final key = _keyFor(quizType);
    final unlocked = await getUnlockedLevels(quizType);
    if (!unlocked.contains(completedLevel.toString())) return null;

    final nextLevel = levels[currentIndex + 1];
    if (!unlocked.contains(nextLevel.toString())) {
      final updated = <int>{
        ...unlocked.map((value) => int.parse(value)),
        nextLevel,
      }.toList()
        ..sort();
      await prefs.setStringList(
        key,
        updated.map((level) => level.toString()).toList(),
      );
```

```
    }
    return nextLevel;
  }

  /// Returns a sorted list of unlocked level numbers as strings for compatibility
  /// with progress saved by earlier versions of the game.
  static Future<List<String>> getUnlockedLevels(String quizType) async {
    final prefs = await SharedPreferences.getInstance();
    final stored = prefs.getStringList(_keyFor(quizType));
    final levels = <int>{};
    for (final value in stored ?? const <String>[]) {
      final parsed = int.tryParse(value);
      if (parsed != null && parsed > 0) levels.add(parsed);
    }
    if (levels.isEmpty) levels.add(1);
    final sorted = levels.toList()..sort();
    return sorted.map((level) => level.toString()).toList();
  }

  static Future<void> resetProgressForQuiz(String quizType) async {
    final prefs = await SharedPreferences.getInstance();
    final key = _keyFor(quizType);
    await prefs.remove(key);
    final trimmedType = quizType.trim();
    final titleCaseType = trimmedType.isEmpty
        ? trimmedType
        : '${trimmedType[0].toUpperCase()}${trimmedType.substring(1).toLowerCase()}';
    for (final legacyKey in <String>{
      'levels__$trimmedType',
      'levels__$titleCaseType',
    }) {
      if (legacyKey != key) await prefs.remove(legacyKey);
    }
  }

  static Future<void> resetLevels(
    String quizType, {
    int totalLevels = 10,
  }) async {
    await resetProgressForQuiz(quizType);
    await initializeLevelsForQuiz(quizType, totalLevels: totalLevels);
  }
}
```

lib/services/profile_functions.dart

36 lignes

```dart
import 'package:shared_preferences/shared_preferences.dart';

class ProfileManager {
  static const String _scoreKey = 'total_score';

  /// Adds earned points to the locally saved score and returns the new total.
  static Future<int> updateScore(int additionalPoints) async {
    if (additionalPoints < 0) {
      throw ArgumentError.value(
        additionalPoints,
        'additionalPoints',
        'ne peut pas être négatif',
      );
    }

    final prefs = await SharedPreferences.getInstance();
    final newTotal = (prefs.getInt(_scoreKey) ?? 0) + additionalPoints;
    await prefs.setInt(_scoreKey, newTotal);
    return newTotal;
  }

  static Future<void> resetScore() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_scoreKey, 0);
  }

  static Future<void> clearScore() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_scoreKey);
  }

  static Future<int> getTotalScore() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getInt(_scoreKey) ?? 0;
  }
}
```

```dart
import 'dart:convert';

import 'package:flutter/foundation.dart' show FlutterError;
import 'package:flutter/services.dart';
import 'package:quiz_game7/models/question.dart';

class QuizContentException implements Exception {
  const QuizContentException(this.message);

  final String message;

  @override
  String toString() => message;
}

class QuestionLoader {
  static String _normalizedQuizType(String quizType) {
    final normalizedType = quizType.trim().toLowerCase();
    if (!RegExp(r'^[a-z0-9_-]+$').hasMatch(normalizedType)) {
      throw ArgumentError.value(quizType, 'quizType', 'identifiant invalide');
    }
    return normalizedType;
  }

  static Future<dynamic> _loadJson(String path) async {
    late final String data;
    try {
      data = await rootBundle.loadString(path);
    } on FlutterError {
      throw QuizContentException(
        'Le fichier « $path » est introuvable. Vérifie les fichiers de questions et pubspec.yaml.',
      );
    }

    try {
      return jsonDecode(data);
    } on FormatException catch (error) {
      throw QuizContentException('JSON invalide dans « $path » : ${error.message}');
    }
  }

  static Future<List<int>> loadAvailableLevels(String quizType) async {
    final type = _normalizedQuizType(quizType);
    final path = 'assets/questions/$type/levels.json';
    final decoded = await _loadJson(path);
    if (decoded is! Map<String, dynamic>) {
      throw QuizContentException('« $path » doit contenir un objet JSON.');
    }

    final rawLevels = decoded['Levels'] ?? decoded['levels'];
    if (rawLevels is! List) {
      throw QuizContentException('Le fichier « $path » doit contenir une liste « Levels ».');
    }

    final levels = <int>[];
    for (final value in rawLevels) {
      final parsed = value is int
          ? value
          : value is String
              ? int.tryParse(value)
              : null;
      if (parsed == null || parsed < 1) {
        throw QuizContentException('« $path » contient un numéro de niveau invalide.');
      }
      if (levels.contains(parsed)) {
        throw QuizContentException('« $path » contient le niveau $parsed en double.');
      }
      levels.add(parsed);
    }

    levels.sort();
    if (levels.isNotEmpty && levels.first != 1) {
      throw QuizContentException('Le premier niveau de « $path » doit être le niveau 1.');
    }
    return levels;
  }

  static Future<List<Question>> loadQuestions(String quizType, int niveau) async {
    if (niveau < 1) {
      throw ArgumentError.value(niveau, 'niveau', 'doit être positif');
    }
    final type = _normalizedQuizType(quizType);
    final path = 'assets/questions/$type/level_$niveau.json';
    final decoded = await _loadJson(path);
    if (decoded is! List) {
      throw QuizContentException('« $path » doit contenir une liste JSON de questions.');
    }

    final questions = <Question>[];
    for (var index = 0; index < decoded.length; index++) {
```

```
        final value = decoded[index];
        if (value is! Map) {
          throw QuizContentException('La question ${index + 1} de « $path » doit être un objet.');
        }
        try {
          questions.add(Question.fromJson(Map<String, dynamic>.from(value)));
        } on FormatException catch (error) {
          throw QuizContentException('Question ${index + 1} de « $path » : ${error.message}');
        }
      }

      if (questions.isEmpty) {
        throw QuizContentException('Aucune question n’est définie dans « $path ».');
      }
      return questions;
    }
}
```

lib/services/quiz_scoring.dart

15 lignes

```dart
import 'package:quiz_game7/models/question.dart';

/// Counts correct answers. A timed-out or unanswered question earns no point.
int calculateQuizScore(List<Question> questions, List<String?> answers) {
  var score = 0;
  for (var index = 0; index < questions.length && index < answers.length; index++) {
    if (answers[index] == questions[index].correctAnswer) score++;
  }
  return score;
}

double calculateQuizAccuracy(List<Question> questions, List<String?> answers) {
  if (questions.isEmpty) return 0;
  return calculateQuizScore(questions, answers) / questions.length;
}
```

lib/widgets/animated_background.dart

32 lignes

```dart
import 'package:flutter/material.dart';

import 'cycling_background.dart';

/// Fond d'écran animé qui alterne les images locales du projet.
class AnimatedBackground extends StatelessWidget {
  const AnimatedBackground({super.key, required this.child});

  static const List<String> _backgroundImages = <String>[
    'assets/backgrounds/bg1.png',
    'assets/backgrounds/bg2.png',
    'assets/backgrounds/bg3.jpeg',
  ];

  final Widget child;

  @override
  Widget build(BuildContext context) {
    return CyclingBackground(
      images: _backgroundImages,
      interval: const Duration(seconds: 10),
      child: Stack(
        fit: StackFit.expand,
        children: <Widget>[
          // Assombrit légèrement les photos pour garder le texte blanc lisible.
          const ColoredBox(color: Color(0x660B1020)),
          child,
        ],
      ),
    );
  }
}
```

```dart
import 'dart:async';

import 'package:flutter/material.dart';

/// Optional asset-based background for screens that provide their own images.
/// An empty image list safely falls back to displaying [child] unchanged.
class CyclingBackground extends StatefulWidget {
  const CyclingBackground({
    super.key,
    required this.child,
    required this.images,
    this.interval = const Duration(seconds: 6),
  });

  final Widget child;
  final List<String> images;
  final Duration interval;

  @override
  State<CyclingBackground> createState() => _CyclingBackgroundState();
}

class _CyclingBackgroundState extends State<CyclingBackground> {
  int _currentIndex = 0;
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    _startCycle();
  }

  @override
  void didUpdateWidget(covariant CyclingBackground oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.images != widget.images || oldWidget.interval != widget.interval) {
      _timer?.cancel();
      _currentIndex = 0;
      _startCycle();
    }
  }

  void _startCycle() {
    if (widget.images.length < 2 || widget.interval <= Duration.zero) return;
    _timer = Timer.periodic(widget.interval, (_) {
      if (!mounted || widget.images.isEmpty) return;
      setState(() => _currentIndex = (_currentIndex + 1) % widget.images.length);
    });
  }

  @override
  void dispose() {
    _timer?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    if (widget.images.isEmpty) return widget.child;
    return Stack(
      fit: StackFit.expand,
      children: <Widget>[
        AnimatedSwitcher(
          duration: const Duration(milliseconds: 700),
          child: Image.asset(
            widget.images[_currentIndex],
            key: ValueKey<String>(widget.images[_currentIndex]),
            fit: BoxFit.cover,
            errorBuilder: (context, error, stackTrace) => const ColoredBox(
              color: Color(0xFF182848),
            ),
          ),
        ),
        widget.child,
      ],
    );
  }
}
```

main.dart

20 lignes

```dart
import 'package:flutter/material.dart';
import 'package:quiz_game7/screens/quiz_type_selection_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Quiz éducatif',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.deepPurple),
      home: const QuizTypeSelectionScreen(), // démarrage direct
    );
  }
}
```

android/app/build.gradle.kts

42 lignes

```
plugins {
    id("com.android.application")
    id("kotlin-android")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-plugin")
}

android {
    namespace = "com.example.quiz_game"
    compileSdk = flutter.compileSdkVersion

    // \u{1F527} Fix : Forcer la version NDK requise par les plugins
    ndkVersion = "27.0.12077973"

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }

    kotlinOptions {
        jvmTarget = JavaVersion.VERSION_11.toString()
    }

    defaultConfig {
        applicationId = "com.example.quiz_game"
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }

    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("debug")
        }
    }
}


flutter {
    source = "../.."
}
```

android/app/src/debug/AndroidManifest.xml

7 lignes

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- The INTERNET permission is required for development. Specifically,
         the Flutter tool needs it to communicate with the running application
         to allow setting breakpoints, to provide hot reload, etc.
    -->
    <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

android/app/src/main/AndroidManifest.xml

45 lignes

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:label="quiz_game"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTop"
            android:taskAffinity=""
            android:theme="@style/LaunchTheme"
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDirection|fontScale|screenLayout|density|uiMode"
            android:hardwareAccelerated="true"
            android:windowSoftInputMode="adjustResize">
            <!-- Specifies an Android theme to apply to this Activity as soon as
                 the Android process has started. This theme is visible to the user
                 while the Flutter UI initializes. After that, this theme continues
                 to determine the Window background behind the Flutter UI. -->
            <meta-data
              android:name="io.flutter.embedding.android.NormalTheme"
              android:resource="@style/NormalTheme"
              />
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <!-- Don't delete the meta-data below.
             This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
        <meta-data
            android:name="flutterEmbedding"
            android:value="2" />
    </application>
    <!-- Required to query activities that can process text, see:
         https://developer.android.com/training/package-visibility and
         https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT.

         In particular, this is used by the Flutter engine in io.flutter.plugin.text.ProcessTextPlugin. -->
    <queries>
        <intent>
            <action android:name="android.intent.action.PROCESS_TEXT"/>
            <data android:mimeType="text/plain"/>
        </intent>
    </queries>
</manifest>
```

```java
package io.flutter.plugins;

import androidx.annotation.Keep;
import androidx.annotation.NonNull;
import io.flutter.Log;

import io.flutter.embedding.engine.FlutterEngine;

/**
 * Generated file. Do not edit.
 * This file is generated by the Flutter tool based on the
 * plugins that support the Android platform.
 */
@Keep
public final class GeneratedPluginRegistrant {
  private static final String TAG = "GeneratedPluginRegistrant";
  public static void registerWith(@NonNull FlutterEngine flutterEngine) {
    try {
      flutterEngine.getPlugins().add(new xyz.luan.audioplayers.AudioplayersPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin audioplayers_android, xyz.luan.audioplayers.AudioplayersPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new io.flutter.plugins.pathprovider.PathProviderPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin path_provider_android, io.flutter.plugins.pathprovider.PathProviderPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin shared_preferences_android, io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin", e);
    }
  }
}
```

android/app/src/main/kotlin/com/example/quiz_game/MainActivity.kt

5 lignes

```
package com.example.quiz_game

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

android/app/src/main/kotlin/com/example/quiz_game7/MainActivity.kt

5 lignes

```
package com.example.quiz_game7

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

android/app/src/main/res/drawable-v21/launch_background.xml

12 lignes

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Modify this file to customize your launch splash screen -->
<layer-list xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="?android:colorBackground" />

    <!-- You can insert your own image assets here -->
    <!-- <item>
        <bitmap
            android:gravity="center"
            android:src="@mipmap/launch_image" />
    </item> -->
</layer-list>
```

android/app/src/main/res/drawable/launch_background.xml

12 lignes

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Modify this file to customize your launch splash screen -->
<layer-list xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="@android:color/white" />

    <!-- You can insert your own image assets here -->
    <!-- <item>
        <bitmap
            android:gravity="center"
            android:src="@mipmap/launch_image" />
    </item> -->
</layer-list>
```

android/app/src/main/res/values-night/styles.xml

18 lignes

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Theme applied to the Android Window while the process is starting when the OS's Dark Mode setting is on -->
    <style name="LaunchTheme" parent="@android:style/Theme.Black.NoTitleBar">
        <!-- Show a splash screen on the activity. Automatically removed when
             the Flutter engine draws its first frame -->
        <item name="android:windowBackground">@drawable/launch_background</item>
    </style>
    <!-- Theme applied to the Android Window as soon as the process has started.
         This theme determines the color of the Android Window while your
         Flutter UI initializes, as well as behind your Flutter UI while its
         running.

         This Theme is only used starting with V2 of Flutter's Android embedding. -->
    <style name="NormalTheme" parent="@android:style/Theme.Black.NoTitleBar">
        <item name="android:windowBackground">?android:colorBackground</item>
    </style>
</resources>
```

android/app/src/main/res/values/styles.xml

18 lignes

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Theme applied to the Android Window while the process is starting when the OS's Dark Mode setting is off -->
    <style name="LaunchTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <!-- Show a splash screen on the activity. Automatically removed when
             the Flutter engine draws its first frame -->
        <item name="android:windowBackground">@drawable/launch_background</item>
    </style>
    <!-- Theme applied to the Android Window as soon as the process has started.
         This theme determines the color of the Android Window while your
         Flutter UI initializes, as well as behind your Flutter UI while its
         running.

         This Theme is only used starting with V2 of Flutter's Android embedding. -->
    <style name="NormalTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <item name="android:windowBackground">?android:colorBackground</item>
    </style>
</resources>
```

android/app/src/profile/AndroidManifest.xml

7 lignes

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- The INTERNET permission is required for development. Specifically,
         the Flutter tool needs it to communicate with the running application
         to allow setting breakpoints, to provide hot reload, etc.
    -->
    <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

android/build.gradle.kts

21 lignes

```
allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

val newBuildDir: Directory = rootProject.layout.buildDirectory.dir("../../build").get()
rootProject.layout.buildDirectory.value(newBuildDir)

subprojects {
    val newSubprojectBuildDir: Directory = newBuildDir.dir(project.name)
    project.layout.buildDirectory.value(newSubprojectBuildDir)
}
subprojects {
    project.evaluationDependsOn(":app")
}

tasks.register<Delete>("clean") {
    delete(rootProject.layout.buildDirectory)
}
```

android/gradle.properties

3 lignes

```
org.gradle.jvmargs=-Xmx8G -XX:MaxMetaspaceSize=4G -XX:ReservedCodeCacheSize=512m -XX:+HeapDumpOnOutOfMemoryError
android.useAndroidX=true
android.enableJetifier=true
```

android/gradle/wrapper/gradle-wrapper.properties

5 lignes

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.10.2-all.zip
```

```
#!/usr/bin/env bash

##############################################################################
##
##  Gradle start up script for UN*X
##
##############################################################################

# Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this script.
DEFAULT_JVM_OPTS=""

APP_NAME="Gradle"
APP_BASE_NAME=`basename "$0"`

# Use the maximum available, or set MAX_FD != -1 to use that value.
MAX_FD="maximum"

warn ( ) {
    echo "$*"
}

die ( ) {
    echo
    echo "$*"
    echo
    exit 1
}

# OS specific support (must be 'true' or 'false').
cygwin=false
msys=false
darwin=false
case "`uname`" in
  CYGWIN* )
    cygwin=true
    ;;
  Darwin* )
    darwin=true
    ;;
  MINGW* )
    msys=true
    ;;
esac

# Attempt to set APP_HOME
# Resolve links: $0 may be a link
PRG="$0"
# Need this for relative symlinks.
while [ -h "$PRG" ] ; do
    ls=`ls -ld "$PRG"`
    link=`expr "$ls" : '.*-> \(.*\)$'`
    if expr "$link" : '/.*' > /dev/null; then
        PRG="$link"
    else
        PRG=`dirname "$PRG"`"/$link"
    fi
done
SAVED="`pwd`"
cd "`dirname \"$PRG\"`/" >/dev/null
APP_HOME="`pwd -P`"
cd "$SAVED" >/dev/null

CLASSPATH=$APP_HOME/gradle/wrapper/gradle-wrapper.jar

# Determine the Java command to use to start the JVM.
if [ -n "$JAVA_HOME" ] ; then
    if [ -x "$JAVA_HOME/jre/sh/java" ] ; then
        # IBM's JDK on AIX uses strange locations for the executables
        JAVACMD="$JAVA_HOME/jre/sh/java"
    else
        JAVACMD="$JAVA_HOME/bin/java"
    fi
    if [ ! -x "$JAVACMD" ] ; then
        die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
else
    JAVACMD="java"
    which java >/dev/null 2>&1 || die "ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH.

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
fi

# Increase the maximum file descriptors if we can.
if [ "$cygwin" = "false" -a "$darwin" = "false" ] ; then
    MAX_FD_LIMIT=`ulimit -H -n`
    if [ $? -eq 0 ] ; then
```

```
        if [ "$MAX_FD" = "maximum" -o "$MAX_FD" = "max" ] ; then
            MAX_FD="$MAX_FD_LIMIT"
        fi
        ulimit -n $MAX_FD
        if [ $? -ne 0 ] ; then
            warn "Could not set maximum file descriptor limit: $MAX_FD"
        fi
    else
        warn "Could not query maximum file descriptor limit: $MAX_FD_LIMIT"
    fi
fi

# For Darwin, add options to specify how the application appears in the dock
if $darwin; then
    GRADLE_OPTS="$GRADLE_OPTS \"-Xdock:name=$APP_NAME\" \"-Xdock:icon=$APP_HOME/media/gradle.icns\""
fi

# For Cygwin, switch paths to Windows format before running java
if $cygwin ; then
    APP_HOME=`cygpath --path --mixed "$APP_HOME"`
    CLASSPATH=`cygpath --path --mixed "$CLASSPATH"`
    JAVACMD=`cygpath --unix "$JAVACMD"`

    # We build the pattern for arguments to be converted via cygpath
    ROOTDIRSRAW=`find -L / -maxdepth 1 -mindepth 1 -type d 2>/dev/null`
    SEP=""
    for dir in $ROOTDIRSRAW ; do
        ROOTDIRS="$ROOTDIRS$SEP$dir"
        SEP="|"
    done
    OURCYGPATTERN="(^($ROOTDIRS))"
    # Add a user-defined pattern to the cygpath arguments
    if [ "$GRADLE_CYGPATTERN" != "" ] ; then
        OURCYGPATTERN="$OURCYGPATTERN|($GRADLE_CYGPATTERN)"
    fi
    # Now convert the arguments - kludge to limit ourselves to /bin/sh
    i=0
    for arg in "$@" ; do
        CHECK=`echo "$arg"|egrep -c "$OURCYGPATTERN" -`
        CHECK2=`echo "$arg"|egrep -c "^-"`                                 ### Determine if an option

        if [ $CHECK -ne 0 ] && [ $CHECK2 -eq 0 ] ; then                    ### Added a condition
            eval `echo args$i`=`cygpath --path --ignore --mixed "$arg"`
        else
            eval `echo args$i`="\"$arg\""
        fi
        i=$((i+1))
    done
    case $i in
        (0) set -- ;;
        (1) set -- "$args0" ;;
        (2) set -- "$args0" "$args1" ;;
        (3) set -- "$args0" "$args1" "$args2" ;;
        (4) set -- "$args0" "$args1" "$args2" "$args3" ;;
        (5) set -- "$args0" "$args1" "$args2" "$args3" "$args4" ;;
        (6) set -- "$args0" "$args1" "$args2" "$args3" "$args4" "$args5" ;;
        (7) set -- "$args0" "$args1" "$args2" "$args3" "$args4" "$args5" "$args6" ;;
        (8) set -- "$args0" "$args1" "$args2" "$args3" "$args4" "$args5" "$args6" "$args7" ;;
        (9) set -- "$args0" "$args1" "$args2" "$args3" "$args4" "$args5" "$args6" "$args7" "$args8" ;;
    esac
fi

# Split up the JVM_OPTS And GRADLE_OPTS values into an array, following the shell quoting and substitution rules
function splitJvmOpts() {
    JVM_OPTS=("$@")
}
eval splitJvmOpts $DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS
JVM_OPTS[${#JVM_OPTS[*]}]="-Dorg.gradle.appname=$APP_BASE_NAME"

exec "$JAVACMD" "${JVM_OPTS[@]}" -classpath "$CLASSPATH" org.gradle.wrapper.GradleWrapperMain "$@"
```

```
@if "%DEBUG%" == "" @echo off
@rem ##########################################################################
@rem
@rem  Gradle startup script for Windows
@rem
@rem ##########################################################################

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this script.
set DEFAULT_JVM_OPTS=

set DIRNAME=%~dp0
if "%DIRNAME%" == "" set DIRNAME=.
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if "%ERRORLEVEL%" == "0" goto init

echo.
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH.
echo.
echo Please set the JAVA_HOME variable in your environment to match the
echo location of your Java installation.

goto fail

:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto init

echo.
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME%
echo.
echo Please set the JAVA_HOME variable in your environment to match the
echo location of your Java installation.

goto fail

:init
@rem Get command-line arguments, handling Windowz variants

if not "%OS%" == "Windows_NT" goto win9xME_args
if "%@eval[2+2]" == "4" goto 4NT_args

:win9xME_args
@rem Slurp the command line arguments.
set CMD_LINE_ARGS=
set _SKIP=2

:win9xME_args_slurp
if "x%~1" == "x" goto execute

set CMD_LINE_ARGS=%*
goto execute

:4NT_args
@rem Get arguments from the 4NT Shell from JP Software
set CMD_LINE_ARGS=%$

:execute
@rem Setup the command line

set CLASSPATH=%APP_HOME%\gradle\wrapper\gradle-wrapper.jar

@rem Execute Gradle
"%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname=%APP_BASE_NAME%" -classpath "%CLASSPATH%" org.gradle.wrapper.GradleWrapperMain %CMD_LINE_ARGS%

:end
@rem End local scope for the variables with windows NT shell
if "%ERRORLEVEL%"=="0" goto mainEnd

:fail
rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code instead of
rem the _cmd.exe /c_ return code!
if  not "" == "%GRADLE_EXIT_CONSOLE%" exit 1
exit /b 1

:mainEnd
if "%OS%"=="Windows_NT" endlocal

:omega
```

android/settings.gradle.kts

25 lignes

```
pluginManagement {
    val flutterSdkPath = run {
        val properties = java.util.Properties()
        file("local.properties").inputStream().use { properties.load(it) }
        val flutterSdkPath = properties.getProperty("flutter.sdk")
        require(flutterSdkPath != null) { "flutter.sdk not set in local.properties" }
        flutterSdkPath
    }

    includeBuild("$flutterSdkPath/packages/flutter_tools/gradle")

    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

plugins {
    id("dev.flutter.flutter-plugin-loader") version "1.0.0"
    id("com.android.application") version "8.7.0" apply false
    id("org.jetbrains.kotlin.android") version "1.8.22" apply false
}

include(":app")
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
fix_android_config.py — aligne android/app/build.gradle(.kts) sur les besoins des plugins.

Le plugin audioplayers exige un minSdk plus élevé que le défaut de Flutter
(minSdk 23 contre 21). Ce script remonte donc la valeur si nécessaire, sans
jamais la baisser, aussi bien dans la syntaxe Groovy que dans la syntaxe
Kotlin DSL utilisée par les projets Flutter récents.

Usage :
    fix_android_config.py [--min-sdk 23] [--compile-sdk 36] [--root <projet>]
"""

from __future__ import annotations

import argparse
import os
import re
import sys

SETTING = re.compile(
    r"(?P<key>minSdk(?:Version)?|compileSdk(?:Version)?)"
    r"(?P<sep>\s*=\s*|\s+)"
    r"(?P<value>flutter\.(?:minSdkVersion|compileSdkVersion)|\d+)"
)


def normalize(key: str) -> str:
    return key.removesuffix("Version").lower()


def find_gradle_file(root: str) -> str | None:
    for relative in ("android/app/build.gradle.kts", "android/app/build.gradle"):
        candidate = os.path.join(root, relative)
        if os.path.isfile(candidate):
            return candidate
    return None


def patch(path: str, targets: dict[str, int]) -> list[str]:
    with open(path, encoding="utf-8") as handle:
        text = handle.read()

    kts = path.endswith(".kts")
    changes: list[str] = []

    def replace(match: re.Match) -> str:
        key = match.group("key")
        key_norm = normalize(key)
        if key_norm not in targets:
            return match.group(0)

        raw_value = match.group("value")
        wanted = targets[key_norm]

        if not raw_value.startswith("flutter."):
            current = int(raw_value)
            if current >= wanted:
                return match.group(0)

        separator = match.group("sep")
        if kts and not separator.strip():
            separator = " = "
        elif not separator.strip():
            separator = " "

        changes.append(f"{key}: {raw_value} -> {wanted}")
        return f"{key}{separator}{wanted}"

    updated = SETTING.sub(replace, text)

    if updated != text:
        with open(path, "w", encoding="utf-8") as handle:
            handle.write(updated)

    return changes


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--root", default=os.environ.get("GITHUB_WORKSPACE", os.getcwd()))
    parser.add_argument("--min-sdk", type=int, default=None)
    parser.add_argument("--compile-sdk", type=int, default=None)
    args = parser.parse_args()

    targets: dict[str, int] = {}
    if args.min_sdk:
        targets["minsdk"] = args.min_sdk
    if args.compile_sdk:
```

```
        targets["compilesdk"] = args.compile_sdk

    if not targets:
        print("[android] aucun ajustement demandé")
        return 0

    gradle_file = find_gradle_file(args.root)
    if gradle_file is None:
        print("::warning::android/app/build.gradle(.kts) introuvable — ajustement ignoré.")
        return 0

    changes = patch(gradle_file, targets)
    relative = os.path.relpath(gradle_file, args.root)
    if changes:
        for change in changes:
            print(f"[android] {relative} — {change}")
    else:
        print(f"[android] {relative} — déjà conforme ({targets})")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
prepare_project.py — prépare le projet Flutter avant la compilation de l'APK.

Appelé par .github/workflows/build-apk.yml.

1. Localise la racine du projet Flutter :
     - le dépôt lui-même si lib/main.dart s'y trouve déjà ;
     - sinon une archive .zip déposée à la racine du dépôt (décompressée) ;
     - sinon un sous-dossier contenant pubspec.yaml + lib/main.dart.
2. Vérifie que le code source est bien présent. Sinon, s'arrête en expliquant
   précisément ce qui manque et comment le fournir.
3. Corrige une contrainte de SDK Dart qui interdit Dart 3
   (ex. « >=2.17.0 <3.0.0 » -> « >=3.0.0 <4.0.0 »).
4. Ajoute une version au pubspec si elle est absente (nécessaire pour l'APK).
5. Retire du pubspec.yaml les assets absents ou en double : flutter refuse de
   démarrer si une seule entrée d'asset n'existe pas sur le disque.
6. Écrit project_root / project_name / app_name dans $GITHUB_OUTPUT.
"""

from __future__ import annotations

import glob
import os
import re
import sys
import zipfile

WORKSPACE = os.path.abspath(os.environ.get("GITHUB_WORKSPACE") or os.getcwd())
EXTRACT_DIR = os.path.join(WORKSPACE, ".projet_extraite")

IGNORED_DIRS = {".git", ".github", ".dart_tool", "build", "node_modules", ".idea", "ios", "web", "windows", "linux", "macos"}
MAX_DEPTH = 3

# Point d'entrée réel d'une application Flutter : lib/main.dart.
# (un main.dart isolé à la racine du dépôt ne suffit pas : flutter ne l'utilise pas)
ENTRY_POINT = os.path.join("lib", "main.dart")

SDK_LINE = re.compile(r'^(?P<indent>[ \t]*)sdk:[ \t]*(?P<quote>["\']?)(?P<value>[^"\'\n#]*)(?P=quote)', re.M)
VERSION_LINE = re.compile(r'^version:[ \t]*\S', re.M)
ASSETS_KEY = re.compile(r'assets:[ \t]*$')
ASSET_ITEM = re.compile(r'-[ \t]+(?P<path>.+?)[ \t]*$')

EXPECTED_STRUCTURE = """\
quiz_game7/
├── lib/                <- tout le code Dart (models, screens, services, widgets)
│   └── main.dart
├── assets/             <- questions JSON, sons, images, logo
├── pubspec.yaml
└── android/            <- optionnel : généré automatiquement s'il est absent
"""


def log(message: str) -> None:
    print(f"[prepare] {message}", flush=True)


def set_output(key: str, value: str) -> None:
    path = os.environ.get("GITHUB_OUTPUT")
    if path:
        with open(path, "a", encoding="utf-8") as handle:
            handle.write(f"{key}={value}\n")
    log(f"sortie : {key} = {value}")


def add_summary(markdown: str) -> None:
    path = os.environ.get("GITHUB_STEP_SUMMARY")
    if path:
        with open(path, "a", encoding="utf-8") as handle:
            handle.write(markdown + "\n")


def has_dart_source(root: str) -> bool:
    return os.path.isfile(os.path.join(root, ENTRY_POINT))


def looks_like_project(path: str) -> bool:
    return os.path.isfile(os.path.join(path, "pubspec.yaml")) and has_dart_source(path)


def has_stray_main() -> bool:
    """Un main.dart isolé à la racine : présent dans ce dépôt, inutilisable par Flutter."""
    return os.path.isfile(os.path.join(WORKSPACE, "main.dart")) and not has_dart_source(WORKSPACE)


def walk_projects(base: str):
    """Parcourt `base` (profondeur limitée) à la recherche d'un projet Flutter."""
    base = os.path.abspath(base)
    for current, dirs, _files in os.walk(base):
```

```
        depth = os.path.relpath(current, base).count(os.sep)
        dirs[:] = [d for d in dirs if d not in IGNORED_DIRS and not d.startswith(".")]
        if current != base and depth < MAX_DEPTH and looks_like_project(current):
            yield current
        if depth >= MAX_DEPTH:
            dirs[:] = []


def extract_archives() -> list[str]:
    """Décompresse les .zip déposés à la racine du dépôt."""
    zones: list[str] = []
    for archive in sorted(glob.glob(os.path.join(WORKSPACE, "*.zip"))):
        target = os.path.join(EXTRACT_DIR, os.path.splitext(os.path.basename(archive))[0])
        log(f"archive détectée : {os.path.basename(archive)} -> décompression dans « {os.path.relpath(target, WORKSPACE)} »")
        os.makedirs(target, exist_ok=True)
        try:
            with zipfile.ZipFile(archive) as zf:
                zf.extractall(target)
        except zipfile.BadZipFile:
            log(f"⚠\u{FE0F}  {os.path.basename(archive)} n'est pas une archive zip valide, ignorée.")
            continue
        zones.append(target)
    return zones


def locate_project() -> tuple[str | None, str]:
    if looks_like_project(WORKSPACE):
        return WORKSPACE, "racine du dépôt"
    for zone in extract_archives():
        if looks_like_project(zone):
            return zone, "archive .zip déposée à la racine"
        for candidate in walk_projects(zone):
            return candidate, "archive .zip déposée à la racine"
    for candidate in walk_projects(WORKSPACE):
        return candidate, "sous-dossier du dépôt"
    return None, ""


def _major(version: str) -> int | None:
    found = re.match(r"\s*(\d+)", version)
    return int(found.group(1)) if found else None


def fix_dart_constraint(text: str) -> tuple[str, str | None]:
    """Autorise Dart 3 quand le pubspec l'interdit encore."""
    for match in SDK_LINE.finditer(text):
        value = match.group("value").strip()
        if ">=" not in value or "flutter" in value:
            continue
        upper = re.search(r"<\s*([0-9][0-9.]*)", value)
        if upper is not None and _major(upper.group(1)) is not None and _major(upper.group(1)) >= 4:
            return text, None
        replacement = f'{match.group("indent")}sdk: ">=3.0.0 <4.0.0"'
        return text.replace(match.group(0), replacement, 1), value
    return text, None


def ensure_version(text: str) -> tuple[str, bool]:
    """Le build Android exige un numéro de version dans le pubspec."""
    if VERSION_LINE.search(text):
        return text, False
    lines = text.splitlines(keepends=True)
    insert_at = 1
    for index, line in enumerate(lines):
        if line.startswith("description:"):
            insert_at = index + 1
            break
    lines.insert(insert_at, "version: 1.0.0+1\n")
    return "".join(lines), True


def prune_assets(text: str, root: str) -> tuple[str, list[str], list[str]]:
    """Supprime les entrées d'assets absentes (flutter refuse de démarrer) et les doublons."""
    kept: list[str] = []
    missing: list[str] = []
    duplicates: list[str] = []
    seen: set[str] = set()
    in_assets = False
    assets_indent = 0

    for line in text.splitlines(keepends=True):
        stripped = line.strip()
        indent = len(line) - len(line.lstrip(" "))

        if not in_assets:
            if ASSETS_KEY.fullmatch(stripped):
                in_assets = True
                assets_indent = indent
            kept.append(line)
            continue
```

```

        if stripped == "" or stripped.startswith("#"):
            kept.append(line)
            continue

        item = ASSET_ITEM.fullmatch(stripped)
        if item and indent > assets_indent:
            entry = item.group("path").strip().strip("\"'")
            if entry in seen:
                duplicates.append(entry)
                continue
            seen.add(entry)
            if entry.endswith("/"):
                exists = os.path.isdir(os.path.join(root, entry))
            else:
                exists = os.path.exists(os.path.join(root, entry))
            if not exists:
                missing.append(entry)
                continue
            kept.append(line)
            continue

        in_assets = False
        kept.append(line)

    return "".join(kept), missing, duplicates


def read_field(text: str, field: str) -> str | None:
    match = re.search(rf"^{field}:[ \t]*(?P<value>.+?)[ \t]*$", text, re.M)
    return match.group("value").strip().strip("\"'") if match else None


def explain_missing() -> None:
    listing = []
    for entry in sorted(os.listdir(WORKSPACE)):
        if entry.startswith(".") and entry not in {".gitignore"}:
            continue
        full = os.path.join(WORKSPACE, entry)
        listing.append(f"  {'\u{1F4C1}' if os.path.isdir(full) else '\u{1F4C4}'} {entry}")
    found = "\n".join(listing) if listing else "  (dépôt vide)"

    hint = ""
    if has_stray_main():
        hint = (
            "\n⚠\u{FE0F}  Un fichier « main.dart » isolé se trouve bien à la racine du dépôt, mais Flutter\n"
            "    n'utilise que « lib/main.dart ». Il faut donc créer le dossier lib/ contenant tout\n"
            "    le code source (models, screens, services, widgets).\n"
        )

    message = f"""Le code source de l'application est introuvable dans le dépôt.

Contenu trouvé à la racine du dépôt :
{found}

Structure attendue (celle de votre schéma d'application) :
{EXPECTED_STRUCTURE}{hint}
Comment fournir le code, au choix :
  1. pousser votre projet depuis votre PC : git add . && git commit -m "Ajout du projet" && git push
  2. glisser-déposer le contenu du projet (dossiers lib/, assets/, android/, pubspec.yaml)
     dans la page GitHub du dépôt -> bouton « Add file » -> « Upload files »
  3. déposer une archive .zip de votre projet à la racine du dépôt (elle sera décompressée
     automatiquement par ce workflow)
Dès que le code est présent, la compilation de l'APK se relance toute seule."""

    print("\n" + "=" * 78, flush=True)
    print(message, flush=True)
    print("=" * 78 + "\n", flush=True)
    print("::error::Code source introuvable (lib/main.dart absent) — voir le détail ci-dessus.", flush=True)

    add_summary(
        "## \u{274C} APK non généré : le code source est absent du dépôt\n\n"
        "Le dépôt ne contient que les fichiers de configuration `pubspec.yaml`, `main.dart` à la racine, etc.\n\n"
        "### Ce qui a été trouvé à la racine\n\n```\n" + found + "\n```\n\n"
        "### Structure attendue\n\n```\n" + EXPECTED_STRUCTURE + "```\n\n"
        "### Comment fournir le code\n\n"
        "1. **Depuis votre PC** : `git add .` puis `git commit -m \"Ajout du projet\"` puis `git push`\n"
        "2. **Via l'interface GitHub** : page du dépôt → *Add file* → *Upload files* (glissez les dossiers "
        "`lib/`, `assets/`, `android/` et le `pubspec.yaml`)\n"
        "3. **En déposant un .zip** du projet à la racine du dépôt : il sera décompressé automatiquement\n\n"
        "La compilation se relance automatiquement dès que le code est présent.\n"
    )


def main() -> int:
    log(f"espace de travail : {WORKSPACE}")
    root, origin = locate_project()

    if root is None:
        explain_missing()
```

```
        return 1

    log(f"projet Flutter trouvé : « {os.path.relpath(root, WORKSPACE)} » (via {origin})")

    pubspec_path = os.path.join(root, "pubspec.yaml")
    with open(pubspec_path, encoding="utf-8") as handle:
        original = handle.read()

    text = original

    text, old_constraint = fix_dart_constraint(text)
    if old_constraint:
        log(f"contrainte Dart corrigée : '{old_constraint}' -> '>=3.0.0 <4.0.0'")

    text, version_added = ensure_version(text)
    if version_added:
        log("aucune version déclarée : ajout de « version: 1.0.0+1 »")

    text, missing, duplicates = prune_assets(text, root)
    for entry in duplicates:
        log(f"asset en double retiré du pubspec : {entry}")
    for entry in missing:
        log(f"asset absent retiré du pubspec : {entry} (fichier introuvable sur le disque)")
    if not missing and not duplicates:
        log("tous les assets déclarés existent : rien à corriger")

    if text != original:
        with open(pubspec_path, "w", encoding="utf-8") as handle:
            handle.write(text)
        log("pubspec.yaml mis à jour")
    else:
        log("pubspec.yaml inchangé")

    name = read_field(text, "name") or "app"
    description = read_field(text, "description") or name
    if description.lower().startswith("a new flutter project"):
        description = name

    dart_files = sum(
        1 for _current, _dirs, files in os.walk(os.path.join(root, "lib")) for name_ in files if name_.endswith(".dart")
    ) if os.path.isdir(os.path.join(root, "lib")) else 0

    set_output("project_root", root)
    set_output("project_name", name)
    set_output("app_name", description)
    set_output("missing_assets", str(len(missing)))

    for folder in ("lib", "assets", "android"):
        present = os.path.isdir(os.path.join(root, folder))
        log(f"dossier {folder}/ : {'présent' if present else 'ABSENT'}")
    log(f"fichiers Dart dans lib/ : {dart_files}")

    if not os.path.isdir(os.path.join(root, "assets")):
        print("::warning::Le dossier assets/ est absent : l'application se compilera mais les questions/sons seront manquants.", flush=True)
    if not os.path.isdir(os.path.join(root, "android")):
        print("::notice::Le dossier android/ est absent : il sera généré automatiquement par 'flutter create'.", flush=True)

    add_summary(
        "## \u{1F527} Projet préparé pour la compilation\n\n"
        f"- Projet détecté : `{os.path.relpath(root, WORKSPACE)}` (via {origin})\n"
        f"- Nom du paquet : `{name}`\n"
        f"- Code Dart : {dart_files} fichier(s) dans `lib/`\n"
        f"- Assets absents retirés du pubspec : {len(missing)}\n"
    )
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

```yaml
name: Construire l'APK

# Se lance automatiquement dès qu'un code est poussé sur main,
# ou manuellement depuis l'onglet « Actions » (bouton « Run workflow »).
on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      flutter_version:
        description: "Version de Flutter à utiliser"
        required: false
        default: "3.29.2"

permissions:
  contents: write

concurrency:
  group: apk-${{ github.ref }}
  cancel-in-progress: true

env:
  FLUTTER_VERSION: ${{ inputs.flutter_version || '3.29.2' }}
  # audioplayers (utilisé par le projet) exige au moins l'API 23.
  MIN_SDK: "23"

jobs:
  apk:
    name: Compilation de l'APK Android
    runs-on: ubuntu-latest
    timeout-minutes: 45

    steps:
      - name: Récupérer le dépôt
        uses: actions/checkout@v4

      - name: Préparer le projet (détection, pubspec, assets)
        id: prepare
        run: python3 .github/scripts/prepare_project.py

      - name: Installer Java 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Installer Flutter ${{ env.FLUTTER_VERSION }}
        uses: subosito/flutter-action@v2
        with:
          flutter-version: ${{ env.FLUTTER_VERSION }}
          channel: stable
          cache: true

      - name: Configurer la plateforme Android
        shell: bash
        env:
          ROOT: ${{ steps.prepare.outputs.project_root }}
          PROJECT_NAME: ${{ steps.prepare.outputs.project_name }}
        run: |
          set -euo pipefail
          cd "$ROOT"

          if [ -d android ]; then
            echo "dossier android/ déjà présent : conservé tel quel."
          else
            echo "::notice::dossier android/ absent : génération via 'flutter create'."
            flutter create --platforms=android --project-name "$PROJECT_NAME" --org com.example .
          fi

          # Remonte minSdk si les plugins l'exigent (jamais vers le bas).
          python3 "$GITHUB_WORKSPACE/.github/scripts/fix_android_config.py" --min-sdk "$MIN_SDK"

      - name: Résoudre les dépendances
        shell: bash
        env:
          ROOT: ${{ steps.prepare.outputs.project_root }}
        run: |
          set -uo pipefail
          cd "$ROOT"

          if flutter pub get; then
            echo "Dépendances résolues (pubspec.lock respecté)."
            exit 0
          fi

          echo "::warning::pubspec.lock inutilisable — nouvelle résolution des dépendances (versions plus récentes)."
          rm -f pubspec.lock
          flutter pub get

          # Des versions plus récentes des plugins peuvent exiger un SDK plus haut.
```

```
          python3 "$GITHUB_WORKSPACE/.github/scripts/fix_android_config.py" --min-sdk "$MIN_SDK" --compile-sdk 36

          SDKMANAGER="$(command -v sdkmanager || true)"
          if [ -z "$SDKMANAGER" ] && [ -n "${ANDROID_SDK_ROOT:-}" ]; then
            SDKMANAGER="$(ls "$ANDROID_SDK_ROOT"/cmdline-tools/*/bin/sdkmanager 2>/dev/null | head -1 || true)"
          fi
          if [ -n "$SDKMANAGER" ]; then
            yes | "$SDKMANAGER" --install "platforms;android-36" >/dev/null 2>&1 || true
          fi

      - name: Compiler l'APK
        id: build
        shell: bash
        env:
          ROOT: ${{ steps.prepare.outputs.project_root }}
        run: |
          set -uo pipefail
          cd "$ROOT"

          if flutter build apk --release; then
            echo "mode=release" >> "$GITHUB_OUTPUT"
          else
            echo "::warning::La compilation en mode release a échoué. Nouvelle tentative en mode debug (APK installable mais non optimisé) — voir la cause exacte dans le journal ci-dessus."
            flutter build apk --debug
            echo "mode=debug" >> "$GITHUB_OUTPUT"
          fi

          ls -lh build/app/outputs/flutter-apk/ ./*/build/app/outputs/flutter-apk/ 2>/dev/null || true

      - name: Publier l'APK dans les Releases
        if: success()
        shell: bash
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          ROOT: ${{ steps.prepare.outputs.project_root }}
          PROJECT_NAME: ${{ steps.prepare.outputs.project_name }}
          APP_NAME: ${{ steps.prepare.outputs.app_name }}
          MODE: ${{ steps.build.outputs.mode }}
        run: |
          set -euo pipefail

          APK_DIR="$ROOT/build/app/outputs/flutter-apk"
          APK_FILE="$(ls "$APK_DIR"/app-release.apk "$APK_DIR"/app-debug.apk 2>/dev/null | head -1)"
          if [ -z "$APK_FILE" ]; then
            echo "::error::Aucun APK produit."
            exit 1
          fi

          OUT="$(pwd)/${PROJECT_NAME}-build${GITHUB_RUN_NUMBER}.apk"
          cp "$APK_FILE" "$OUT"
          ls -lh "$OUT"

          TAG="apk-latest"
          gh release delete "$TAG" --yes --cleanup-tag --repo "$GITHUB_REPOSITORY" 2>/dev/null || true
          gh release create "$TAG" \
            --repo "$GITHUB_REPOSITORY" \
            --target "$GITHUB_SHA" \
            --title "APK — build #${GITHUB_RUN_NUMBER} (${MODE})" \
            --notes "APK de « ${APP_NAME} » compilé automatiquement par GitHub Actions.

          - Commit : \`${GITHUB_SHA}\`
          - Branche : \`${GITHUB_REF_NAME}\`
          - Mode de compilation : **${MODE}**
          - Fichier : \`$(basename "$OUT")\`" \
            --latest \
            "$OUT"

          echo "::notice::APK disponible dans l'onglet Releases (tag ${TAG})."

      - name: Enregistrer l'APK comme artifact
        if: success()
        uses: actions/upload-artifact@v4
        with:
          name: apk-${{ steps.build.outputs.mode }}-build${{ github.run_number }}
          path: ${{ steps.prepare.outputs.project_root }}/build/app/outputs/flutter-apk/*.apk
          if-no-files-found: error
          retention-days: 30

      - name: Récapitulatif
        if: success()
        shell: bash
        env:
          MODE: ${{ steps.build.outputs.mode }}
          ROOT: ${{ steps.prepare.outputs.project_root }}
        run: |
          {
            echo "## \u{2705} APK généré"
            echo ""
            echo "| | |"
            echo "|---|---|"
```

```
            echo "| Projet | \`${ROOT#${GITHUB_WORKSPACE}/}\` |"
            echo "| Mode | \`${MODE}\` |"
            echo "| Flutter | \`${FLUTTER_VERSION}\` |"
            echo "| Build | #${GITHUB_RUN_NUMBER} |"
            echo ""
            echo "### Où récupérer l'APK"
            echo ""
            echo "1. **Onglet Releases** de ce dépôt, release \`apk-latest\` → téléchargement direct, ou"
            echo "2. **Bas de cette page**, section *Artifacts* → \`apk-${MODE}-build${GITHUB_RUN_NUMBER}\`."
            echo ""
            echo "Installation sur le téléphone : autoriser « Installer des applications inconnues » pour le navigateur ou le gestionnaire de fichiers, puis ouvrir le fichier .apk."
          } >> "$GITHUB_STEP_SUMMARY"

      - name: Expliquer l'échec
        if: failure()
        shell: bash
        run: |
          {
            echo "## \u{274C} La compilation a échoué"
            echo ""
            echo "Ouvrez l'étape en rouge ci-dessus : la cause exacte y est indiquée."
            echo ""
            echo "Causes les plus fréquentes :"
            echo ""
            echo "- **Code source absent** : le dépôt ne contient pas encore \`lib/\` et \`assets/\`."
            echo "- **Erreur de code Dart** : le message indique le fichier et la ligne concernés."
            echo "- **Asset manquant** : un fichier listé dans \`pubspec.yaml\` n'existe pas."
            echo "- **Version de Flutter** : relancez le workflow avec une autre version via *Run workflow*."
          } >> "$GITHUB_STEP_SUMMARY"
```

.gitignore

45 lignes

```
# Miscellaneous
*.class
*.log
*.pyc
*.swp
.DS_Store
.atom/
.build/
.buildlog/
.history
.svn/
.swiftpm/
migrate_working_dir/

# IntelliJ related
*.iml
*.ipr
*.iws
.idea/

# The .vscode folder contains launch configuration and tasks you configure in
# VS Code which you may wish to be included in version control, so this line
# is commented out by default.
#.vscode/

# Flutter/Dart/Pub related
**/doc/api/
**/ios/Flutter/.last_build_id
.dart_tool/
.flutter-plugins
.flutter-plugins-dependencies
.pub-cache/
.pub/
/build/

# Symbolication related
app.*.symbols

# Obfuscation related
app.*.map.json

# Android Studio will place build artifacts here
/android/app/debug
/android/app/profile
/android/app/release
```

analysis_options.yaml

31 lignes

```yaml
# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in the UI of Dart-enabled
# IDEs (https://dart.dev/tools#ides-and-editors). The analyzer can also be
# invoked from the command line by running `flutter analyze`.

# The following line activates a set of recommended lints for Flutter apps,
# packages, and plugins designed to encourage good coding practices.
analyzer:
  errors:
    depend_on_referenced_packages: ignore
include: package:flutter_lints/flutter.yaml

linter:
  # The lint rules applied to this project can be customized in the
  # section below to disable rules from the `package:flutter_lints/flutter.yaml`
  # included above or to enable additional rules. A list of all available lints
  # and their documentation is published at https://dart.dev/lints.
  #
  # Instead of disabling a lint rule for the entire project in the
  # section below, it can also be suppressed for a single line of code
  # or a specific dart file by using the `// ignore: name_of_lint` and
  # `// ignore_for_file: name_of_lint` syntax on the line or in the file
  # producing the lint.
  rules:
    # avoid_print: false  # Uncomment to disable the `avoid_print` rule
    # prefer_single_quotes: true  # Uncomment to enable the `prefer_single_quotes` rule

# Additional information about this file can be found at
# https://dart.dev/guides/language/analysis-options
```

pubspec.yaml

67 lignes

```yaml
name: quiz_game7
description: A new Flutter project.

environment:
  sdk: ">=2.17.0 <3.0.0"

dependencies:
  flutter:
    sdk: flutter
  # Autres dépendances utiles à ton projet
  cupertino_icons: ^1.0.2
  shared_preferences: ^2.2.2
  audioplayers: ^6.4.0

dev_dependencies:
    flutter_test:
      sdk: flutter
    flutter_lints: ^3.0.1
flutter:

  uses-material-design: true

  assets:
    - assets/logo.png
    - assets/questions/
    - assets/images/
    - assets/questions/coran/levels.json
    - assets/questions/sira/levels.json
    - assets/questions/arabe/levels.json
    - assets/questions/histoire/levels.json
    - assets/questions/sahaba/levels.json
    - assets/questions/hijra/levels.json
    - assets/questions/terme/levels.json
    - assets/questions/general/levels.json
    - assets/questions/coran/level_1.json
    - assets/questions/coran/level_2.json
    - assets/questions/coran/level_3.json
    - assets/questions/coran/level_4.json
    - assets/questions/sira/level_1.json
    - assets/questions/sira/level_2.json
    - assets/questions/sira/level_3.json
    - assets/questions/arabe/level_1.json
    - assets/questions/arabe/level_3.json
    - assets/questions/arabe/level_3.json
    - assets/questions/histoire/level_1.json
    - assets/questions/sahaba/level_1.json
    - assets/questions/sahaba/level_2.json
    - assets/questions/sahaba/level_3.json
    - assets/questions/hijra/level_1.json
    - assets/questions/terme/level_1.json
    - assets/questions/terme/level_2.json
    - assets/questions/terme/level_3.json
    - assets/questions/terme/level_4.json
    - assets/questions/terme/level_5.json
    - assets/questions/terme/level_6.json
    - assets/questions/general/level_1.json
    - assets/questions/general/level_1.json
    - assets/questions/general/level_1.json
    - assets/questions/general/level_2.json
    - assets/questions/general/level_3.json
    - assets/questions/general/level_4.json
    - assets/questions/general/level_5.json
    - assets/sounds/correct.mp3
    - assets/sounds/wrong.mp3
    - assets/backgrounds/bg1.png
    - assets/backgrounds/bg2.png
    - assets/backgrounds/bg3.jpeg
```